

Análisis de Agrupación con KNN y Market Basket

Identificación de similitudes y patrones de consumo

Autor: RobertScience Data Consulting
Proyecto: Práctica M30 – Machine Learning & Pattern Mining
Herramienta: Python / Jupyter Notebook
Entorno: Visual Studio Code

Descripción del proyecto

El objetivo de este análisis es aplicar técnicas de **aprendizaje automático y minería de datos** para resolver dos problemas fundamentales en ciencia de datos:

1. **Identificación de similitud entre observaciones** mediante el algoritmo de *K vecinos más cercanos (KNN)* aplicado a un conjunto de datos de vinos.
2. **Detección de patrones de consumo** mediante el análisis de *Market Basket*, con el fin de identificar relaciones entre productos frecuentemente comprados en conjunto.

A través del uso de estas técnicas, se busca comprender cómo los datos pueden utilizarse para generar información valiosa que apoye la toma de decisiones en contextos reales, como recomendaciones de productos o análisis de comportamiento del consumidor.

Este tipo de análisis es ampliamente utilizado en sectores como retail, marketing y analítica avanzada, donde la identificación de patrones y similitudes representa una ventaja competitiva.

Importación de librerías

En esta sección se importan las librerías necesarias para el análisis de datos, incluyendo herramientas para manipulación de datos, estandarización de variables, aplicación del algoritmo KNN y análisis de patrones de consumo mediante reglas de asociación.

```
In [17]: import pandas as pd
import numpy as np

from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

Carga del conjunto de datos

En esta sección se realiza la carga del dataset de vinos desde un archivo local previamente almacenado dentro del entorno de trabajo.

El dataset contiene información sobre diversas características fisicoquímicas de diferentes muestras de vino, las cuales serán utilizadas para aplicar técnicas de análisis de similitud mediante el algoritmo KNN.

El uso de archivos locales permite tener un mayor control sobre los datos, asegurando la disponibilidad del dataset durante todo el proceso de análisis y evitando dependencias externas.

```
In [18]: df = pd.read_csv(r"D:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\dataTarea
df.head()
```

Out[18]:

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavanoids	Nonflav
0	14.23	1.71	2.43	15.6	127	2.80	3.06	
1	13.20	1.78	2.14	11.2	100	2.65	2.76	
2	13.16	2.36	2.67	18.6	101	2.80	3.24	
3	14.37	1.95	2.50	16.8	113	3.85	3.49	
4	13.24	2.59	2.87	21.0	118	2.80	2.69	

```
In [19]: # Verificar columnas del dataset
print("Columnas del dataset:")
print(df.columns)

# Mantener solo variables numéricas (evita errores en KNN)
df = df.select_dtypes(include=[np.number])

# Validar que exista la columna Alcohol
if 'Alcohol' not in df.columns:
    raise ValueError("La columna 'Alcohol' no existe en el dataset")
```

Columnas del dataset:
Index(['Alcohol', 'Malic_Acid', 'Ash', 'Ash_Alcanity', 'Magnesium',
 'Total_Phenols', 'Flavanoids', 'Nonflavanoid_Phenols',
 'Proanthocyanins', 'Color_Intensity', 'Hue', 'OD280', 'Proline'],
 dtype='str')

En esta sección se realizó la carga del conjunto de datos utilizando un archivo local almacenado dentro del entorno de trabajo.

El dataset fue leído mediante la función `read_csv` de la librería Pandas, lo que permitió convertir el archivo en un DataFrame estructurado para su análisis.

El uso de rutas locales garantiza la estabilidad del flujo de trabajo, evitando posibles fallos asociados a la disponibilidad de fuentes externas.

Finalmente, se visualizaron las primeras filas del dataset con el objetivo de validar que los datos se hayan cargado correctamente y para obtener una primera aproximación a la estructura del conjunto de datos.

Exploración inicial de los datos

Antes de aplicar cualquier modelo de aprendizaje automático, es fundamental comprender la estructura y características del conjunto de datos.

En esta etapa se realiza un análisis exploratorio inicial con el objetivo de:

- Identificar el número de registros y variables
- Revisar los tipos de datos
- Detectar posibles valores faltantes
- Analizar estadísticas descriptivas básicas

Este análisis permite entender la naturaleza de los datos y tomar decisiones informadas para su posterior procesamiento.

```
In [20]: # Información general del dataset
df.info()
```

```
# Estadísticas descriptivas
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Alcohol               178 non-null   float64
1   Malic_Acid            178 non-null   float64
2   Ash                   178 non-null   float64
3   Ash_Alcanity          178 non-null   float64
4   Magnesium             178 non-null   int64   
5   Total_Phenols         178 non-null   float64
6   Flavanoids            178 non-null   float64
7   Nonflavanoid_Phenols  178 non-null   float64
8   Proanthocyanins       178 non-null   float64
9   Color_Intensity       178 non-null   float64
10  Hue                   178 non-null   float64
11  OD280                 178 non-null   float64
12  Proline               178 non-null   int64   
dtypes: float64(11), int64(2)
memory usage: 18.2 KB
```

Out[20]:

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavai
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.00
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.02
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.99
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.34
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.20
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.13
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.87
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.08

En esta sección se realizó una exploración inicial del conjunto de datos con el fin de comprender su estructura y características principales.

A través del método `info()` , se pudo observar el número total de registros, el tipo de datos de cada variable y la presencia o ausencia de valores nulos. Esto permite identificar si es necesario realizar procesos adicionales de limpieza o transformación.

Por otro lado, mediante el uso de `describe()` , se obtuvieron estadísticas descriptivas como la media, desviación estándar, valores mínimos y máximos para cada variable numérica. Estas métricas permiten tener una mejor comprensión de la distribución de los datos.

En conjunto, este análisis proporciona una base sólida para la aplicación de técnicas de aprendizaje automático, asegurando que los datos se encuentren en condiciones adecuadas para su procesamiento.

Preprocesamiento de los datos

Antes de aplicar el algoritmo KNN, es necesario realizar un proceso de estandarización de las variables.

El algoritmo de K vecinos más cercanos se basa en el cálculo de distancias entre observaciones, por lo que las variables con escalas mayores pueden influir de manera desproporcionada en los resultados.

Para evitar este problema, se aplica una transformación que permite que todas las variables tengan una media de 0 y una desviación estándar de 1, asegurando que cada característica tenga el mismo peso dentro del análisis.

```
In [21]: scaler = StandardScaler()

X_scaled = scaler.fit_transform(df)

X_scaled[:5]

Out[21]: array([[ 1.51861254, -0.5622498 ,  0.23205254, -1.16959318,  1.91390522,
  0.80899739,  1.03481896, -0.65956311,  1.22488398,  0.25171685,
  0.36217728,  1.84791957,  1.01300893],
 [ 0.24628963, -0.49941338, -0.82799632, -2.49084714,  0.01814502,
  0.56864766,  0.73362894, -0.82071924, -0.54472099, -0.29332133,
  0.40605066,  1.1134493 ,  0.96524152],
 [ 0.19687903,  0.02123125,  1.10933436, -0.2687382 ,  0.08835836,
  0.80899739,  1.21553297, -0.49840699,  2.13596773,  0.26901965,
  0.31830389,  0.78858745,  1.39514818],
 [ 1.69154964, -0.34681064,  0.4879264 , -0.80925118,  0.93091845,
  2.49144552,  1.46652465, -0.98187536,  1.03215473,  1.18606801,
 -0.42754369,  1.18407144,  2.33457383],
 [ 0.29570023,  0.22769377,  1.84040254,  0.45194578,  1.28198515,
  0.80899739,  0.66335127,  0.22679555,  0.40140444, -0.31927553,
  0.36217728,  0.44960118, -0.03787401]])
```

En esta sección se llevó a cabo el proceso de estandarización de los datos mediante el uso de la clase `StandardScaler`.

Este procedimiento transforma las variables originales para que tengan una media de 0 y una desviación estándar de 1, lo cual es fundamental en algoritmos basados en distancia como KNN.

Sin este paso, variables con valores más grandes podrían dominar el cálculo de distancias, generando resultados sesgados.

Finalmente, se visualizaron algunas de las observaciones transformadas con el objetivo de verificar que el proceso se haya aplicado correctamente.

Aplicación del modelo KNN

En esta sección se implementa el algoritmo de K vecinos más cercanos (KNN) con el objetivo de identificar observaciones similares dentro del conjunto de datos.

El modelo se entrena utilizando los datos previamente estandarizados, permitiendo calcular distancias entre observaciones de manera adecuada.

Para este análisis se considera un total de 5 vecinos más cercanos, de acuerdo con lo solicitado en la práctica.

```
In [22]: knn = NearestNeighbors(n_neighbors=5)

knn.fit(X_scaled)
```

```
Out[22]: ▾ NearestNeighbors ⓘ ?
          ► Parameters
```

En esta sección se implementó el modelo de K vecinos más cercanos utilizando la clase `NearestNeighbors`.

El modelo fue entrenado con los datos previamente estandarizados, lo que permite realizar comparaciones entre observaciones basadas en distancias de manera consistente.

El parámetro `n_neighbors=5` indica que el modelo buscará los cinco registros más similares a una nueva observación, lo cual es clave para cumplir con el objetivo del análisis planteado.

Este modelo será utilizado posteriormente para identificar los vinos más cercanos a un registro específico definido en la siguiente etapa.

Definición del vino de referencia

En esta sección se define un nuevo registro que representa un vino con características específicas.

Este registro será utilizado como punto de referencia para identificar los vinos más similares dentro del dataset mediante el modelo KNN previamente entrenado.

Las características del vino fueron proporcionadas en el planteamiento del problema y corresponden a variables fisicoquímicas del conjunto de datos.

```
In [23]: nuevo_vino = np.array([[14, 2, 2.5, 16, 115, 3, 2.5, 0.4, 2, 9, 1, 3.5, 800]])  
  
nuevo_vino_scaled = scaler.transform(nuevo_vino)
```

```
d:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\.venv\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names  
  warnings.warn(
```

En esta sección se definió un nuevo registro con valores específicos correspondientes a las características de un vino.

Posteriormente, este registro fue transformado utilizando el mismo proceso de estandarización aplicado al conjunto de datos original. Esto es fundamental para asegurar que las comparaciones realizadas por el modelo KNN sean consistentes.

De esta manera, el nuevo vino se encuentra en la misma escala que los datos utilizados para entrenar el modelo, lo que permite obtener resultados confiables en la identificación de similitudes.

Identificación de los vecinos más cercanos

En esta sección se utilizan los datos del vino de referencia para identificar los 5 vinos más similares dentro del dataset.

El modelo KNN previamente entrenado permite calcular las distancias entre el nuevo registro y las observaciones existentes, retornando aquellos registros que presentan mayor similitud.

El objetivo es analizar específicamente el contenido de alcohol de estos vinos para posteriormente calcular su valor promedio.

```
In [24]: distancias, indices = knn.kneighbors(nuevo_vino_scaled)  
  
print("Distancias de los vecinos más cercanos:")  
print(distancias)  
  
vecinos = df.iloc[indices[0]]  
  
vecinos
```

```
Distancias de los vecinos más cercanos:  
[[1.9397578  2.02044456 2.34431352 2.39949199 2.41913661]]
```

Out[24]:

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavanoids	Nonfla
40	13.56	1.71	2.31	16.2	117	3.15	3.29	
56	14.22	1.70	2.30	16.3	118	3.20	3.00	
49	13.94	1.73	2.27	17.4	108	2.88	3.54	
0	14.23	1.71	2.43	15.6	127	2.80	3.06	
20	14.06	1.63	2.28	16.0	126	3.00	3.17	

En esta sección se utilizó el modelo KNN para identificar los cinco vinos más similares al registro definido previamente.

El método `kneighbors()` devuelve tanto las distancias como los índices de los registros más cercanos. Posteriormente, se utilizaron estos índices para extraer los datos correspondientes desde el DataFrame original.

El resultado es un subconjunto de datos que contiene únicamente los vinos más similares, lo que permite analizar directamente sus características y compararlas con el vino de referencia.

Cálculo del contenido de alcohol

En esta sección se analiza el contenido de alcohol de cada uno de los vinos identificados como vecinos más cercanos.

Posteriormente, se calcula el promedio de estos valores con el objetivo de estimar el nivel de alcohol representativo para vinos con características similares.

In [25]:

```
print("Contenido de alcohol de los vinos más similares:\n")
print(vecinos['Alcohol'])

print("\nDetalle individual:\n")
for i, row in vecinos.iterrows():
    print(f"Vino {i} - Alcohol: {row['Alcohol']}")

alcohol_promedio = vecinos['Alcohol'].mean()

print("\nPromedio de alcohol:", alcohol_promedio)
```

Contenido de alcohol de los vinos más similares:

```
40    13.56
56    14.22
49    13.94
0     14.23
20    14.06
Name: Alcohol, dtype: float64
```

Detalle individual:

```
Vino 40 - Alcohol: 13.56
Vino 56 - Alcohol: 14.22
Vino 49 - Alcohol: 13.94
Vino 0 - Alcohol: 14.23
Vino 20 - Alcohol: 14.06
```

Promedio de alcohol: 14.002

En esta sección se extrajo el contenido de alcohol correspondiente a cada uno de los vinos identificados como más similares.

El análisis de estos valores permite observar cómo varía esta característica dentro del grupo de vecinos cercanos.

Posteriormente, se calculó el promedio del contenido de alcohol, lo que proporciona una estimación representativa basada en similitud, cumpliendo con el objetivo planteado en el problema.

Este resultado es clave, ya que refleja cómo el uso de algoritmos de aprendizaje automático puede apoyar en la estimación de valores a partir de datos similares.

Análisis de Market Basket

En esta sección se realiza un análisis de patrones de consumo mediante la técnica conocida como Market Basket Analysis.

Este enfoque permite identificar combinaciones de productos que frecuentemente son adquiridos en conjunto dentro de un conjunto de transacciones.

El objetivo es detectar relaciones entre productos que puedan ser útiles para estrategias comerciales, como recomendaciones o promociones cruzadas.

```
In [26]: transactions = [
    ['bread', 'butter', 'wine', 'bananas', 'coffee', 'carrots'],
    ['tomatoes', 'onions', 'cheese', 'milk', 'potatoes'],
    ['beer', 'chips', 'asparagus', 'salsa', 'milk', 'apples'],
    ['olive oil', 'bread', 'butter', 'tomatoes', 'steak', 'carrots'],
    ['tomatoes', 'onions', 'chips', 'wine', 'ketchup', 'orange juice'],
    ['bread', 'butter', 'beer', 'chips', 'milk'],
    ['butter', 'tomatoes', 'carrots', 'coffee', 'sugar'],
    ['tomatoes', 'onions', 'cheese', 'milk', 'potatoes'],
    ['bread', 'butter', 'ketchup', 'coffee', 'chicken wings'],
    ['butter', 'beer', 'chips', 'asparagus', 'apples'],
    ['tomatoes', 'onions', 'beer', 'chips', 'milk', 'coffee']
]
```

En esta sección se definió un conjunto de transacciones que representa compras realizadas por diferentes clientes.

Cada transacción contiene una lista de productos adquiridos en conjunto, lo que permite analizar patrones de co-ocurrencia entre artículos.

Este tipo de estructura es fundamental para aplicar técnicas de análisis de canasta de mercado, ya que permite identificar relaciones entre productos dentro de múltiples compras.

Este código está diseñado para funcionar con cualquier lista de compras que tenga el mismo formato (lista de listas).

Esto permite reutilizar el análisis en distintos escenarios sin necesidad de modificar la lógica del programa, facilitando su aplicación en contextos reales.

Transformación de los datos

Para aplicar algoritmos de análisis de patrones, es necesario transformar las transacciones en un formato estructurado.

En este caso, se convierte la lista de productos en una matriz binaria donde cada columna representa un producto y cada fila una transacción, indicando la presencia o ausencia de cada elemento.

```
In [27]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

df_market = pd.DataFrame(te_array, columns=te.columns_)

df_market.head()
```

Out[27]:

	apples	asparagus	bananas	beer	bread	butter	carrots	cheese	chicken wings	chips	...	...
0	False	False	True	False	True	True	True	False	False	False	...	False
1	False	False	False	False	False	False	False	True	False	False	...	False
2	True	True	False	True	False	False	False	False	False	True	...	False
3	False	False	False	False	True	True	True	False	False	False	...	False
4	False	False	False	False	False	False	False	False	False	True	...	False

5 rows × 22 columns



En esta sección se transformaron las transacciones en una estructura tabular mediante el uso de la clase `TransactionEncoder`.

El resultado es una matriz binaria donde cada fila representa una transacción y cada columna un producto, indicando con valores booleanos si el producto fue adquirido o no.

Esta transformación es necesaria para aplicar algoritmos de minería de patrones, ya que estos requieren datos en formato estructurado para identificar relaciones entre variables.

Identificación de patrones frecuentes

Se aplica el algoritmo Apriori para identificar combinaciones de productos que aparecen con frecuencia dentro de las transacciones.

Estas combinaciones permiten detectar patrones relevantes en el comportamiento de compra de los clientes.

In [28]:

```
frequent_items = apriori(df_market, min_support=0.2, use_colnames=True)

frequent_items
```


Out[28]:

	support	itemsets
0	0.363636	frozenset({beer})
1	0.363636	frozenset({bread})
2	0.545455	frozenset({butter})
3	0.272727	frozenset({carrots})
4	0.454545	frozenset({chips})
5	0.363636	frozenset({coffee})
6	0.454545	frozenset({milk})
7	0.363636	frozenset({onions})
8	0.545455	frozenset({tomatoes})
9	0.363636	frozenset({chips, beer})
10	0.272727	frozenset({milk, beer})
11	0.363636	frozenset({bread, butter})
12	0.272727	frozenset({carrots, butter})
13	0.272727	frozenset({coffee, butter})
14	0.272727	frozenset({milk, chips})
15	0.272727	frozenset({milk, onions})
16	0.272727	frozenset({milk, tomatoes})
17	0.363636	frozenset({tomatoes, onions})
18	0.272727	frozenset({milk, chips, beer})
19	0.272727	frozenset({milk, tomatoes, onions})

En esta sección se aplicó el algoritmo Apriori con el objetivo de identificar conjuntos de productos que aparecen con frecuencia en las transacciones.

El parámetro `min_support` define el umbral mínimo de frecuencia para considerar un conjunto como relevante dentro del análisis.

El resultado permite identificar patrones de consumo basados en la repetición de combinaciones de productos.

Generación de reglas de asociación

A partir de los patrones frecuentes identificados, se generan reglas de asociación que permiten establecer relaciones entre productos.

Estas reglas indican la probabilidad de que un producto sea comprado dado que otro ya fue adquirido.

In [29]:

```
rules = association_rules(frequent_items, metric="confidence", min_threshold=0.5)

rules

rules.sort_values(by=['confidence', 'lift'], ascending=False).head()
```

Out[29]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	
18	frozenset({milk, chips})	frozenset({beer})	0.272727	0.363636	0.272727	1.0	2.7%
24	frozenset({milk, tomatoes})	frozenset({onions})	0.272727	0.363636	0.272727	1.0	2.7%
1	frozenset({beer})	frozenset({chips})	0.363636	0.454545	0.363636	1.0	2.2%
19	frozenset({milk, beer})	frozenset({chips})	0.272727	0.454545	0.272727	1.0	2.2%
4	frozenset({bread})	frozenset({butter})	0.363636	0.545455	0.363636	1.0	1.8%

Interpretación de resultados

A partir de las reglas de asociación obtenidas, se pueden identificar patrones de consumo relevantes.

Por ejemplo, productos como "bread" y "butter" suelen aparecer juntos en múltiples transacciones, lo que indica una fuerte relación de compra conjunta. De igual manera, combinaciones como "beer" y "chips" reflejan hábitos de consumo comunes.

Estas relaciones pueden ser utilizadas para implementar estrategias de venta cruzada (cross-selling), recomendaciones de productos y optimización de promociones comerciales.

En esta sección se generaron reglas de asociación a partir de los conjuntos frecuentes identificados previamente.

Estas reglas permiten analizar relaciones entre productos, evaluando métricas como la confianza, que indica la probabilidad de que un producto sea adquirido cuando otro ya está presente en la transacción.

Este tipo de análisis es ampliamente utilizado en entornos comerciales para mejorar estrategias de venta y recomendación de productos.

Conclusión

El análisis realizado permitió aplicar técnicas de aprendizaje automático y minería de datos para resolver problemas relacionados con similitud y patrones de consumo.

Mediante el uso del algoritmo KNN, se identificaron vinos con características similares, lo que permitió estimar el contenido de alcohol de un nuevo registro a partir de sus vecinos más cercanos.

Por otro lado, el análisis de Market Basket permitió identificar relaciones entre productos que frecuentemente son adquiridos en conjunto, proporcionando información valiosa sobre el comportamiento de compra.

En conjunto, estos resultados demuestran la utilidad de las técnicas de análisis de datos para generar información relevante que puede ser aplicada en la toma de decisiones dentro de entornos reales.